

Lab6_TinyML_Anomaly_Detection:

Section 1) Edge Impulse Results:

Public edge impulse anomaly project link:

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

I believe we are just using the Nano's accelerometer for XYZ portion of the board where we can get all the movement from the motion orientation.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

There are many different names of anatomy input features starting from accX RMS, accX Peak 1 height, or Peak 3 heights and accZ peak 2 freq or accX spectr5al power from .1 to .5 Hz which are being used to train the classifier. So, we are passing RMS values, peak freq, peak heights and other power in .1-.5 frequency bands for our project. There are around 33 input features

3. NN classifier outputs: State the outputs of the NN classifier.

There are only two outputs to the NN classifier of one nominal and another OFF.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

We are passing three types of Height, RMS and power freq into the detector features where we are using four features in total. They are accX RMS, accX peak 1 height, accY Spectral Power 2-5 Hz, accZ spectral power 2-5 Hz

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

```
Windows PowerShell X Windows PowerShell X + v
starting inferencing in 2 seconds...
sampling...
Timing: DSP 44 ms, inference 564 us, anomaly 6 ms, postprocessing 49 us
Classification predictions:
  nominal: 0.468750
  off: 0.531250
nomaly prediction: 11.765993
starting inferencing in 2 seconds...
sampling...
Timing: DSP 44 ms, inference 564 us, anomaly 6 ms, postprocessing 49 us
Classification predictions:
  nominal: 0.937500
  off: 0.062500
nomaly prediction: 22.358704
starting inferencing in 2 seconds...
sampling...
Timing: DSP 44 ms, inference 609 us, anomaly 6 ms, postprocessing 49 us
Classification predictions:
  nominal: 0.214844
  off: 0.785156
nomaly prediction: 33.499680
starting inferencing in 2 seconds...
sampling...
Timing: DSP 44 ms, inference 583 us, anomaly 6 ms, postprocessing 49 us
Classification predictions:
  nominal: 0.683594
  off: 0.316406
nomaly prediction: 17.099819
starting inferencing in 2 seconds...
sampling...
Timing: DSP 44 ms, inference 584 us, anomaly 6 ms, postprocessing 49 us
Classification predictions:
  nominal: 0.960937
  off: 0.039063
nomaly prediction: 50.987038
starting inferencing in 2 seconds...
sampling...
Timing: DSP 44 ms, inference 583 us, anomaly 6 ms, postprocessing 49 us
Classification predictions:
  nominal: 0.289063
  off: 0.710937
nomaly prediction: 600.017151
starting inferencing in 2 seconds...
```

Tried to move around the board from distance to rotation and sometimes kept in one motion to make the offset and nominal value low.

6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

I think the model is most reliable when the classifier's confident is higher than average but I don't think we should always trust the result coming form the model as there has been some conflict on where the classifier and anomaly detector were wrong and wasn't able get us the right answer or had conflicting results between them which isn't reliable but sometimes when the board is stationary it is mostly reliable.

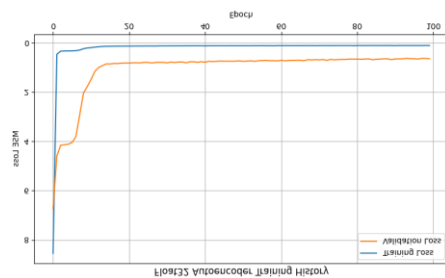
Section 2: Autoencoder Model Results

Include clear and labeled screenshots of the following:

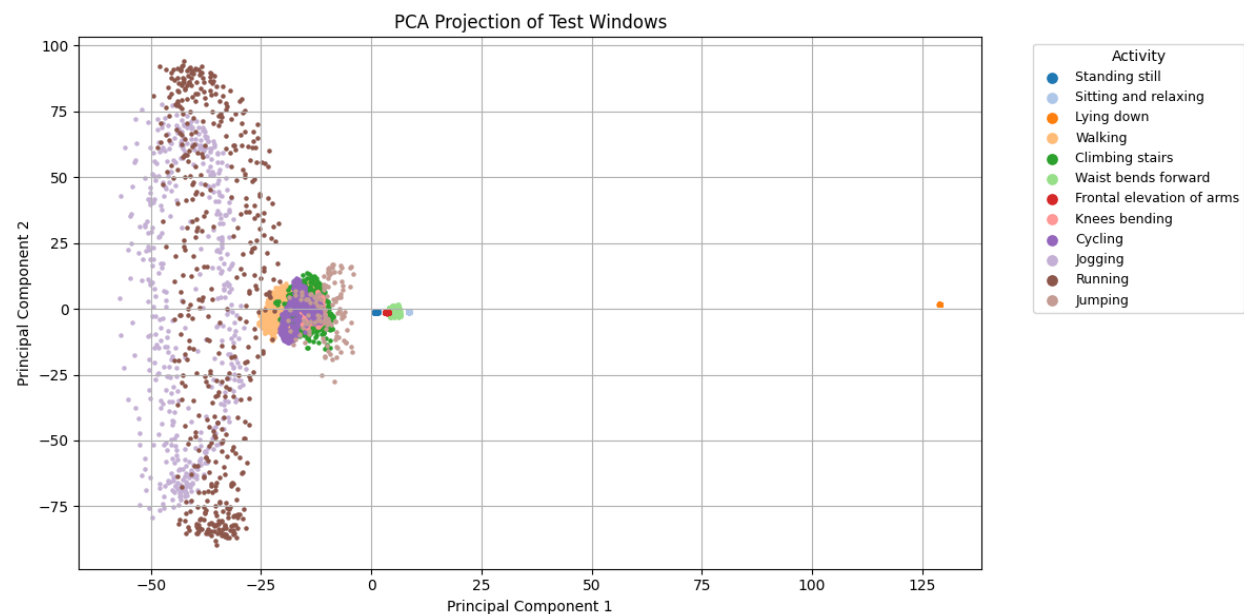
- Training and validation loss curves

```
# Units matter for deployment: the inbuilt accelerometer is recorded in m/s^2.
print("Feature range (m/s^2): min = %.3f, max = %.3f" % (X_all.min(), X_all.max()))
✓ 0.4s

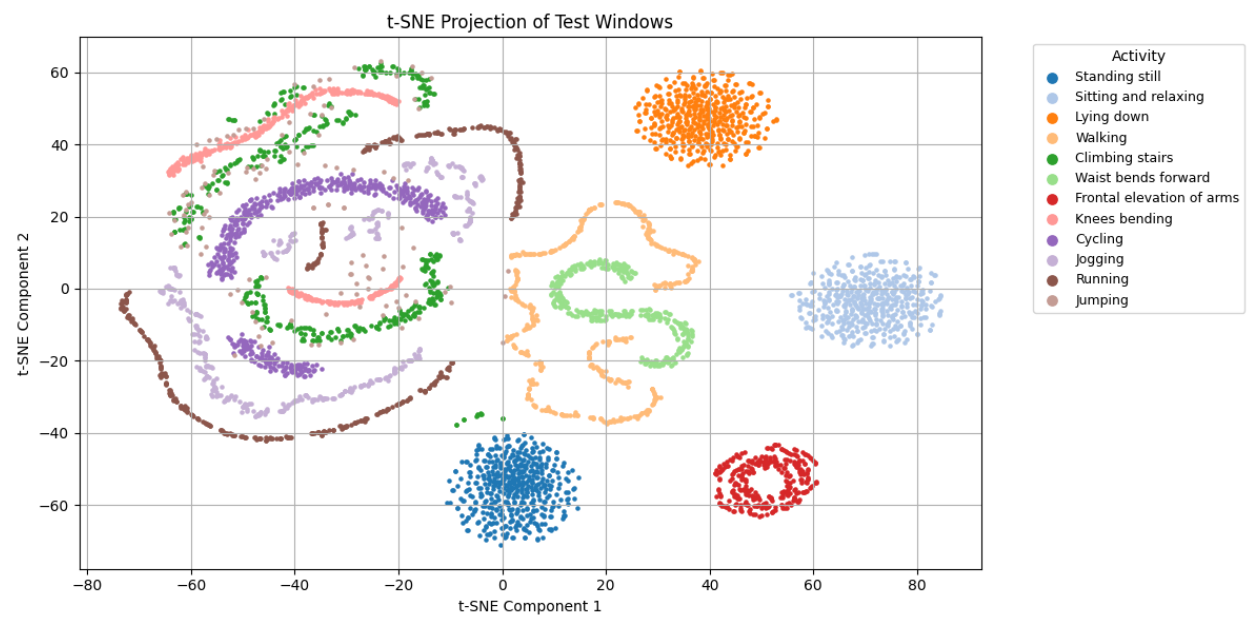
X_train shape: (21350, 300)
X_test shape: (8479, 300)
y_train shape: (21350,)
y_test shape: (8479,)
Training activity distribution: [2051 2051 2051 2051 2051 1442 1370 1513 2051 2051 2051 617]
Testing activity distribution: [823 823 823 823 823 562 531 593 823 823 823 209]
Training binary distribution [normal, anomaly]: [10478 10872]
Testing binary distribution [normal, anomaly]: [4155 4324]
Feature range (m/s^2): min = -22.091, max = 24.599
```



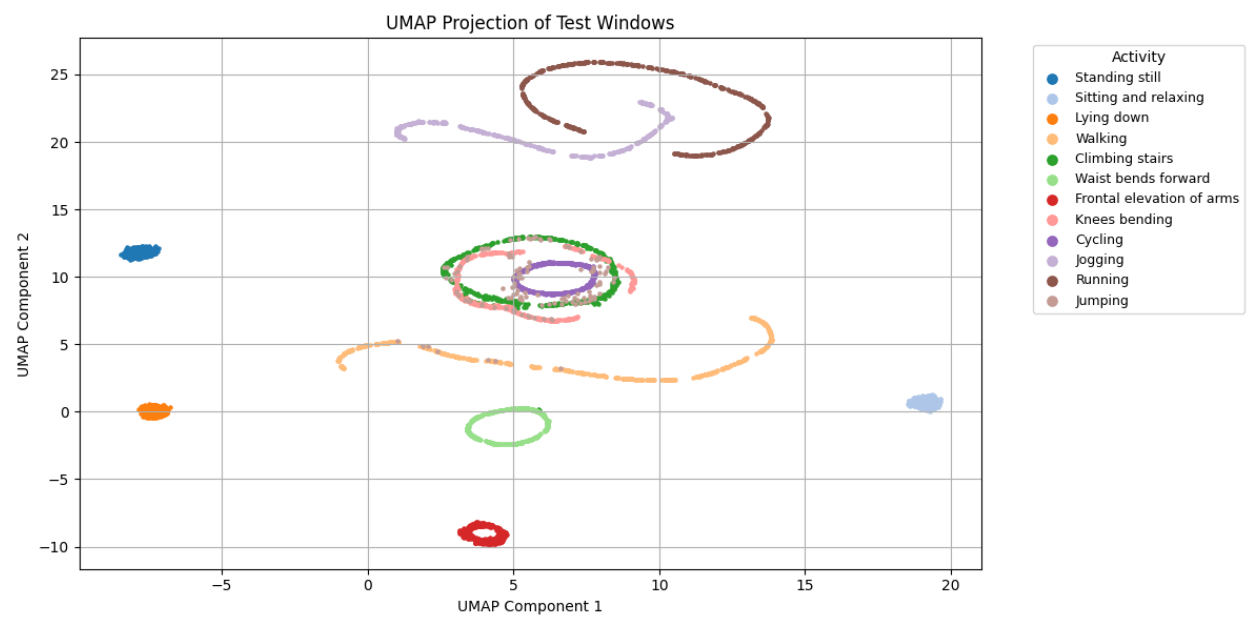
- PCA visualization



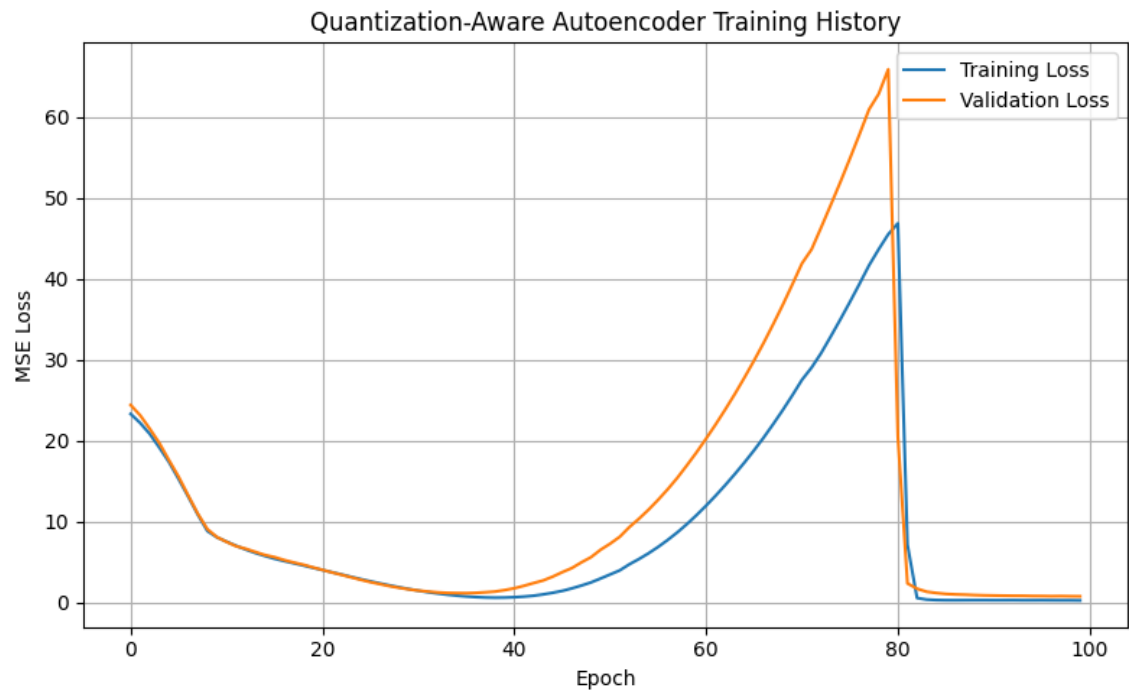
- t-SNE visualization



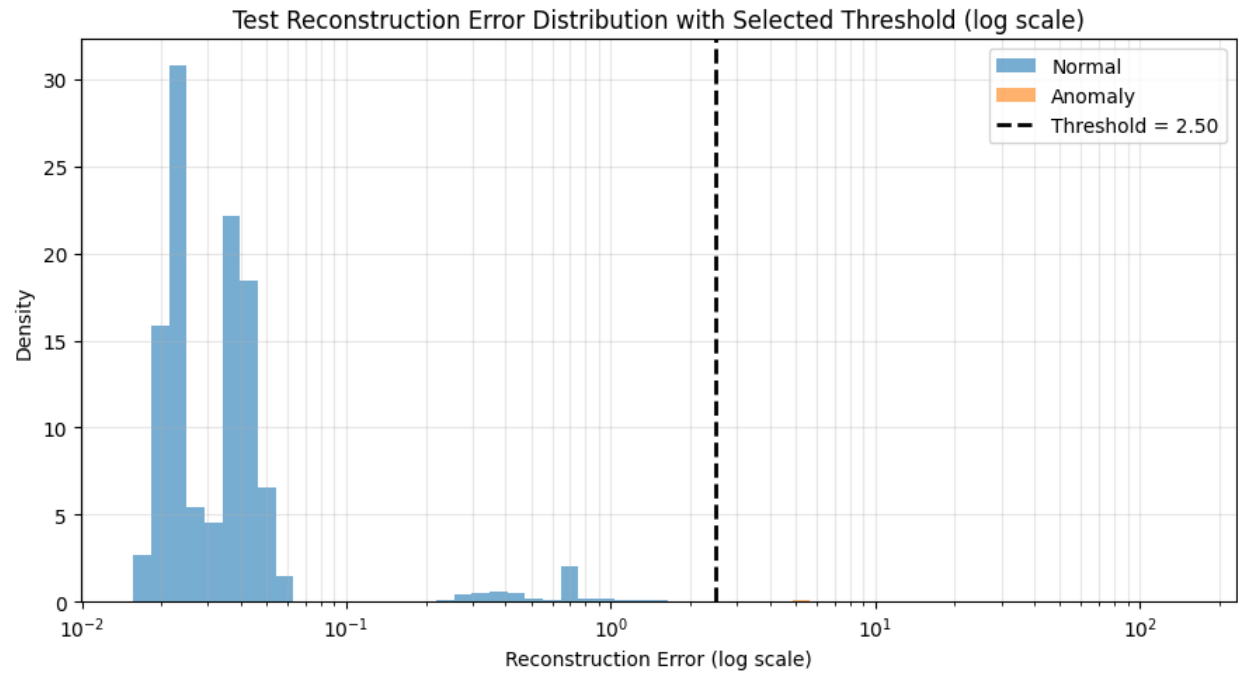
- UMAP visualization



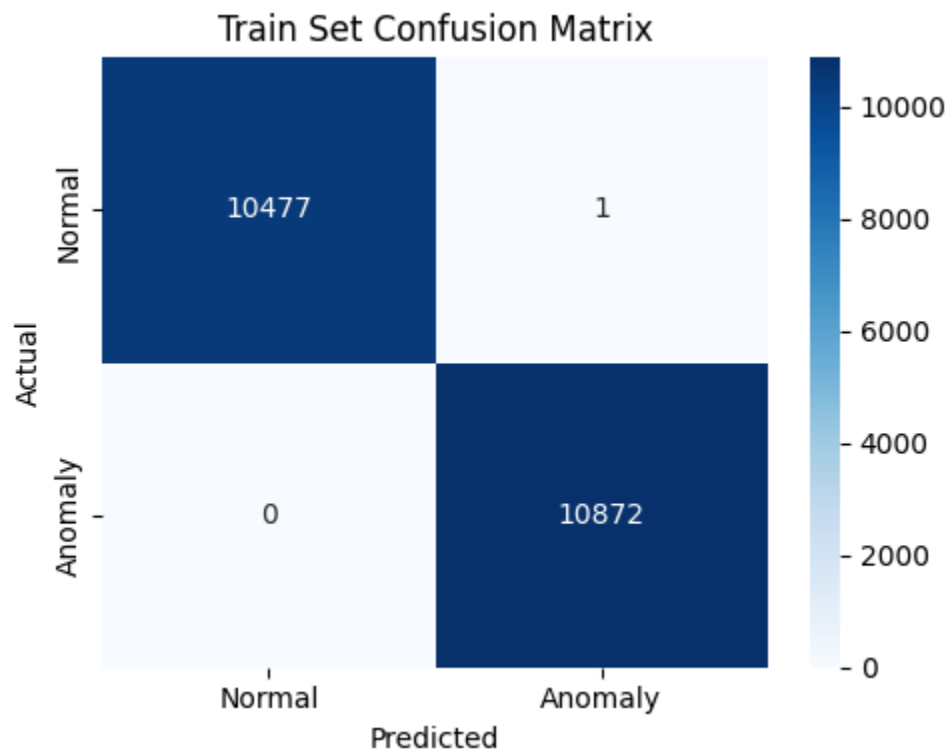
- Reconstruction error distribution

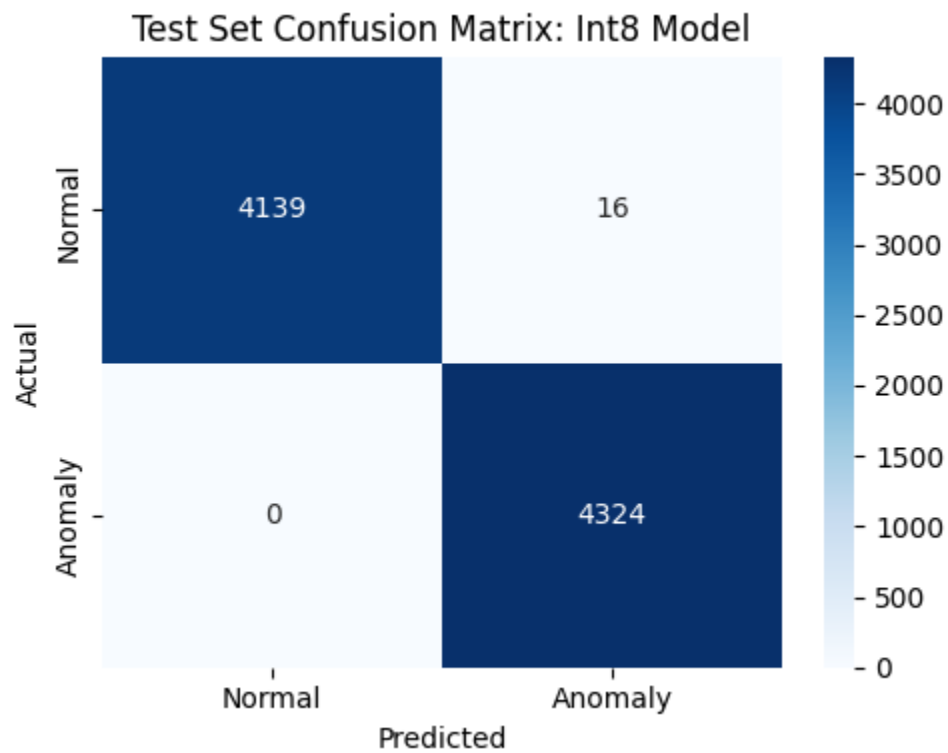
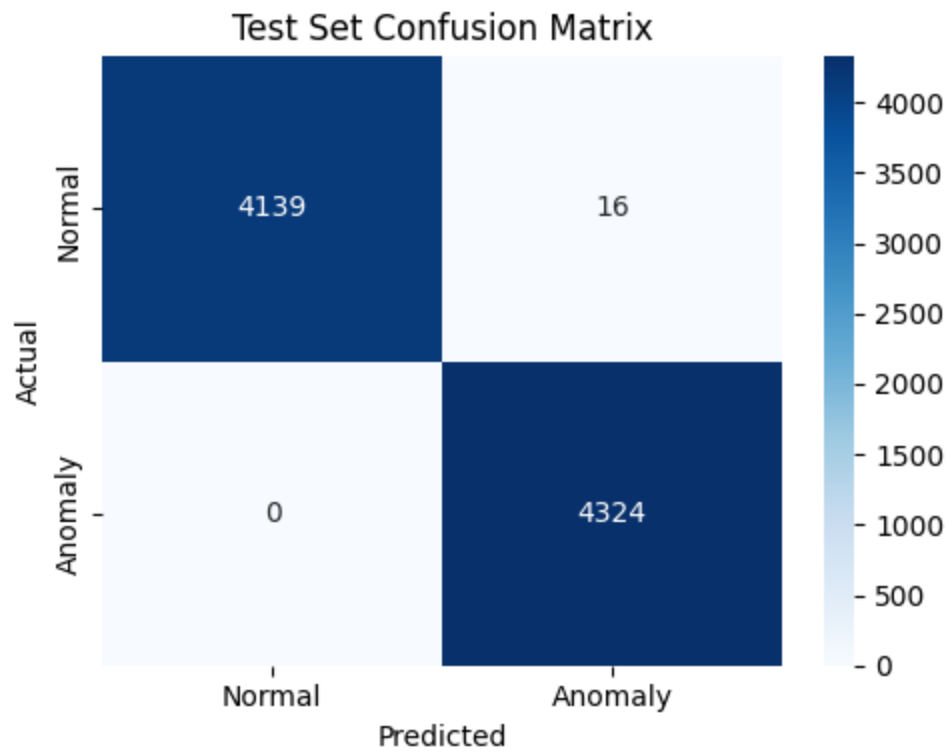


- Selected anomaly threshold



- Confusion matrix





- Classification report or evaluation metrics

Representable input range: [-11.878, 8.868] m/s²

Training data range : [-22.091, 24.599] m/s²

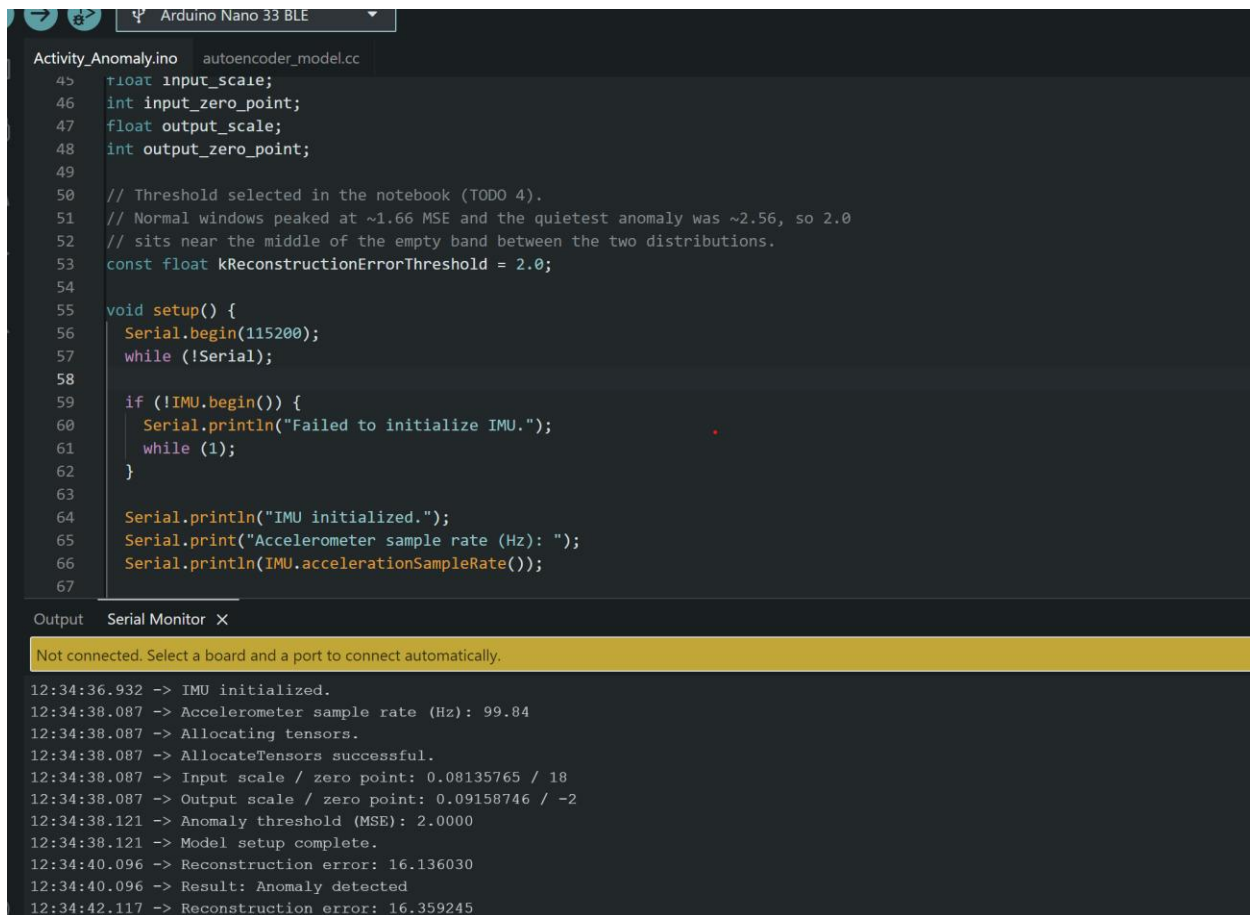
Estimated tensor memory: 21.90 KB

model size comparison

Float32 TFLite : 82.87 KB

Full int8 TFLite : 24.46 KB

- Arduino serial monitor output showing reconstruction error and normal/anomaly detection results



The screenshot displays the Arduino IDE interface. The top pane shows the code for `Activity_Anomaly.ino`, which includes headers for `autoencoder_model.cc` and various variables for input/output scales and zero points. It also defines a reconstruction error threshold of 2.0. The `setup()` function initializes the serial port at 115200 baud and the IMU. The bottom pane shows the serial monitor output, which includes timestamps and messages for IMU initialization, tensor allocation, model setup, and reconstruction error calculations. The output shows two instances of "Anomaly detected" with reconstruction errors of 16.136030 and 16.359245.

```
Activity_Anomaly.ino autoencoder_model.cc
45 float input_scale;
46 int input_zero_point;
47 float output_scale;
48 int output_zero_point;
49
50 // Threshold selected in the notebook (TODO 4).
51 // Normal windows peaked at ~1.66 MSE and the quietest anomaly was ~2.56, so 2.0
52 // sits near the middle of the empty band between the two distributions.
53 const float kReconstructionErrorThreshold = 2.0;
54
55 void setup() {
56   Serial.begin(115200);
57   while (!Serial);
58
59   if (!IMU.begin()) {
60     Serial.println("Failed to initialize IMU.");
61     while (1);
62   }
63
64   Serial.println("IMU initialized.");
65   Serial.print("Accelerometer sample rate (Hz): ");
66   Serial.println(IMU.accelerationSampleRate());
67
Output Serial Monitor X
Not connected. Select a board and a port to connect automatically.

12:34:36.932 -> IMU initialized.
12:34:38.087 -> Accelerometer sample rate (Hz): 99.84
12:34:38.087 -> Allocating tensors.
12:34:38.087 -> AllocateTensors successful.
12:34:38.087 -> Input scale / zero point: 0.08135765 / 18
12:34:38.087 -> Output scale / zero point: 0.09158746 / -2
12:34:38.121 -> Anomaly threshold (MSE): 2.0000
12:34:38.121 -> Model setup complete.
12:34:40.096 -> Reconstruction error: 16.136030
12:34:40.096 -> Result: Anomaly detected
12:34:42.117 -> Reconstruction error: 16.359245
```

Section 3: Discussion Questions

Answer the following briefly:

- What threshold did you choose for anomaly detection, and why?

I used a threshold based on the reconstruction-error distribution of known-normal data, typically the 99th percentile so it can keep the overall false alarm low for the system.

- How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder learns to reconstruct normal patterns well. Normal activity gives small reconstruction errors and for anomalous it can be learnt output of the normal learning method to produce a larger error with anomaly score.

- What information from the Python notebook must be preserved when deploying the model to Arduino?

We would have to peruse the model in c++ code file so we can transfer the trained model architecture with weights/activations and other thresholds as Arduino would use that data bytes and can't train anything due to resource constraints.

- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

It is important as the notebook can compute the parameters and train a model with a big dataset and it has to be the same preprocessing due to certain input/feature assumptions. So if Arduino board utilizes a different scaling or ordering, it would see make harder for the model for what kind of data the sensor is getting or if its even a data or instruction.

- What are the main limitations of this anomaly detection method when deployed on a microcontroller?

I think the limited resources to capture the anomaly detection isn't very accurate when I was using it on my board as it stating anomaly detect many times with just being in stationery and accuracy might've also dropped from the quantization with limited data to train from.